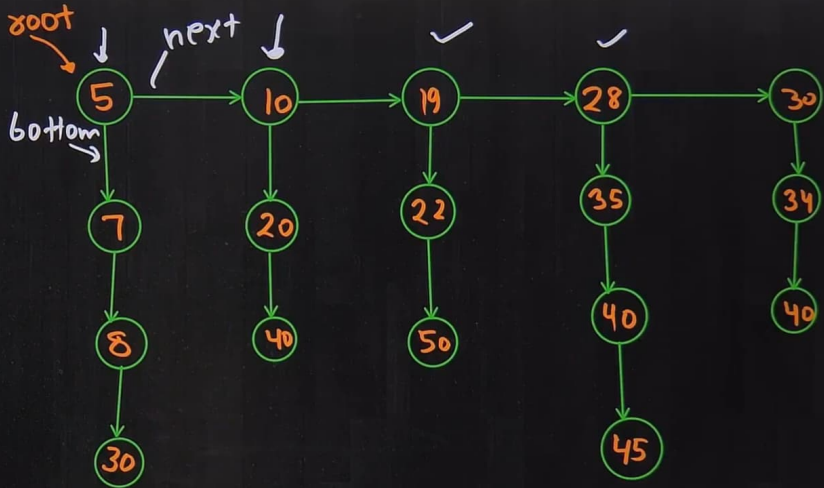
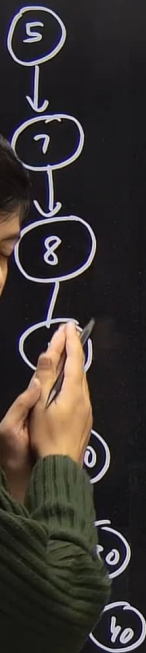


Flattening a Linked List

535 837

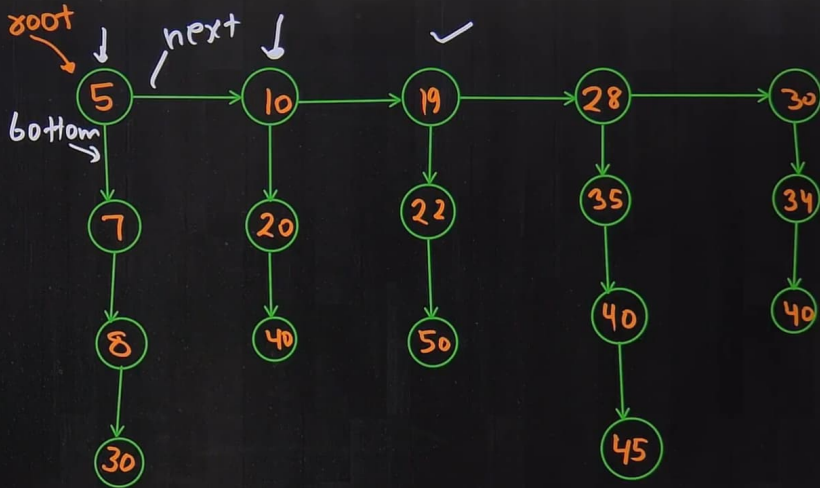
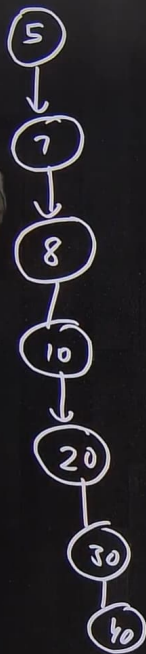
16:17

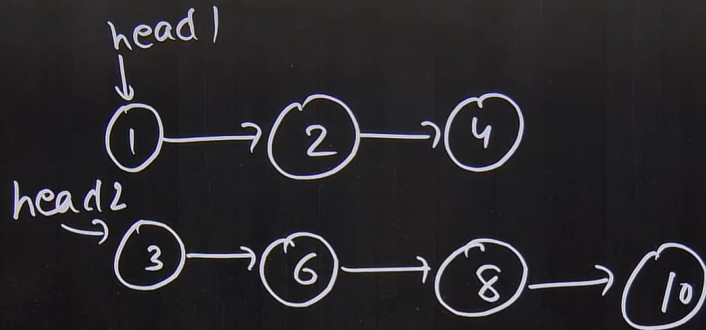


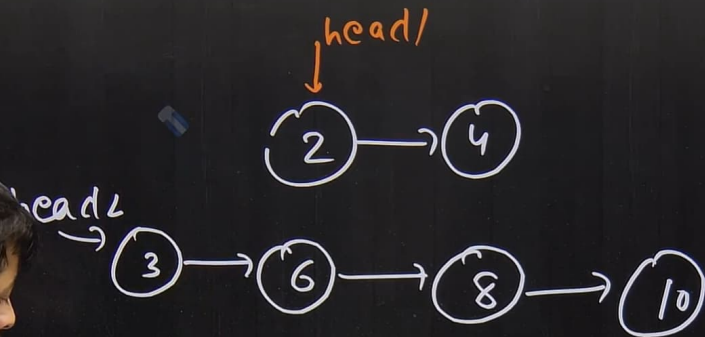
Flattening a Linked List

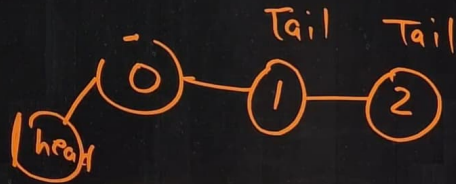
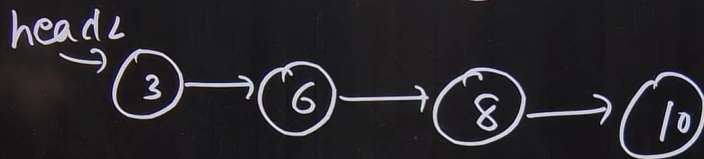
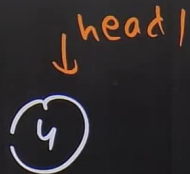
535 837

16:17



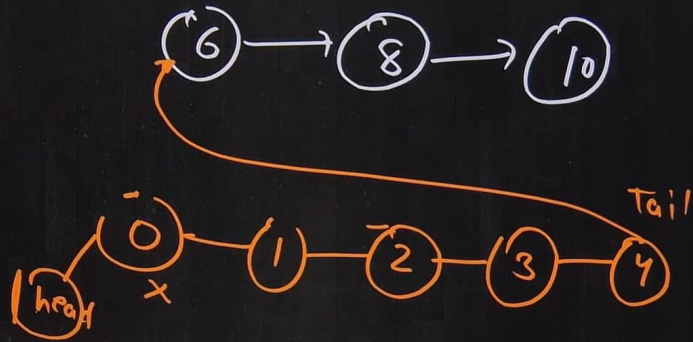






head1

head2



Node * merge(Node* head1, Node* head2)

{
Node * head = new Node(0);
Node * Tail = head;

While(head1 && head2)
{

if (head1->data <= head2->data)

{
Tail->bottom = head1;

head1 = head1->bottom;

Tail = Tail->bottom;

Tail->bottom = NULL;

}

Node * merge(Node* head1, Node* head2)

```
{  
    Node * head = new Node(0);  
    Node * Tail = head;
```

```
    while(head1 && head2)
```

```
    {  
        if (head1->data <= head2->data)
```

```
        {  
            Tail->bottom = head1;
```

```
            head1 = head1->bottom;
```

```
            Tail = Tail->bottom;
```

```
        }  
        Tail->bottom = NULL;
```

```
    }  
    else
```

```
    {  
        Tail->bottom = head2;
```

```
        head2 = head2->bottom;
```

```
        Tail = Tail->bottom;
```

```
    }  
    Tail->bottom = NULL;
```


Node * merge(Node* head1, Node* head2)

{
Node * head = new Node(0);
Node * Tail = head;

if (head1)

Tail->bottom = head1;

else

Tail->bottom = head2;

return head->next;

}

while (head1 && head2)

{ if (head1->data <= head2->data)

{ Tail->bottom = head1;

head1 = head1->bottom;

Tail = Tail->bottom;

Tail->bottom = NULL;

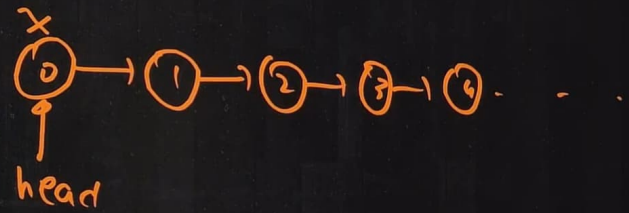
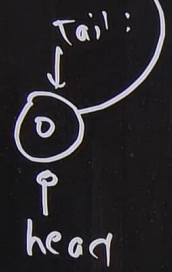
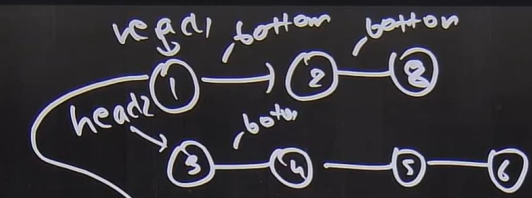
else

{ Tail->bottom = head2;

head2 = head2->bottom;

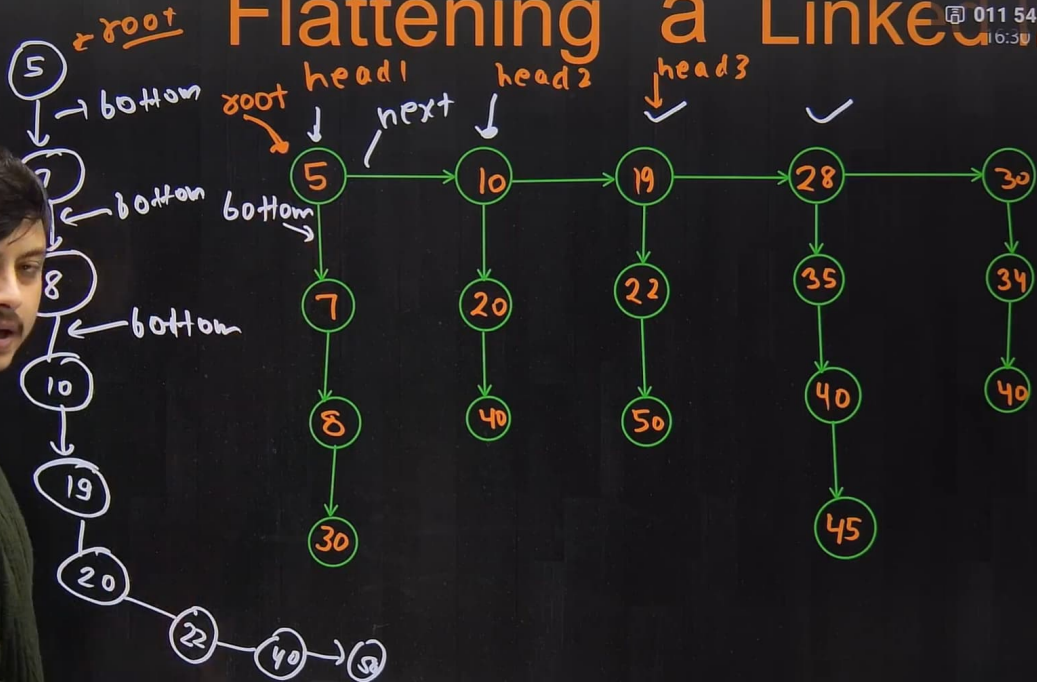
Tail = Tail->bottom;

Tail->bottom = NULL;

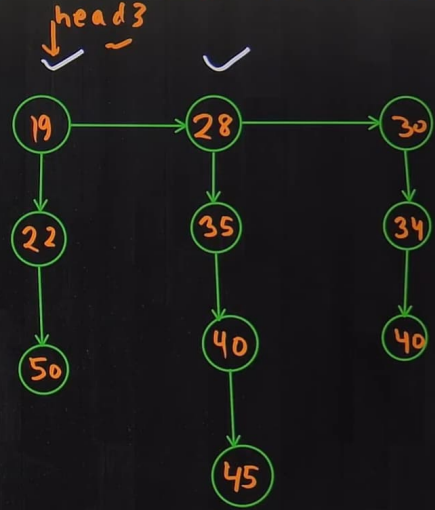
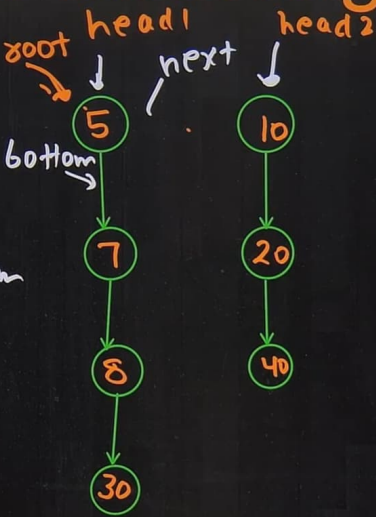
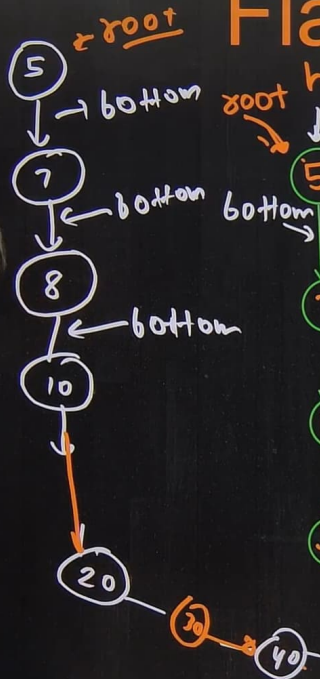


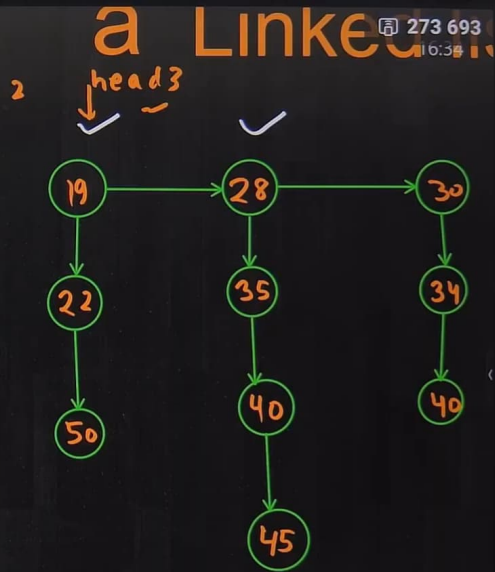
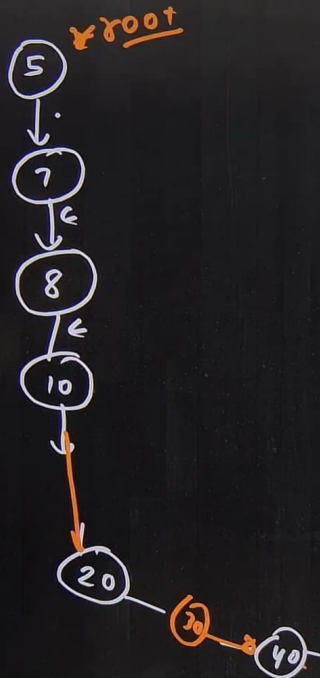
Flattening a Linked List

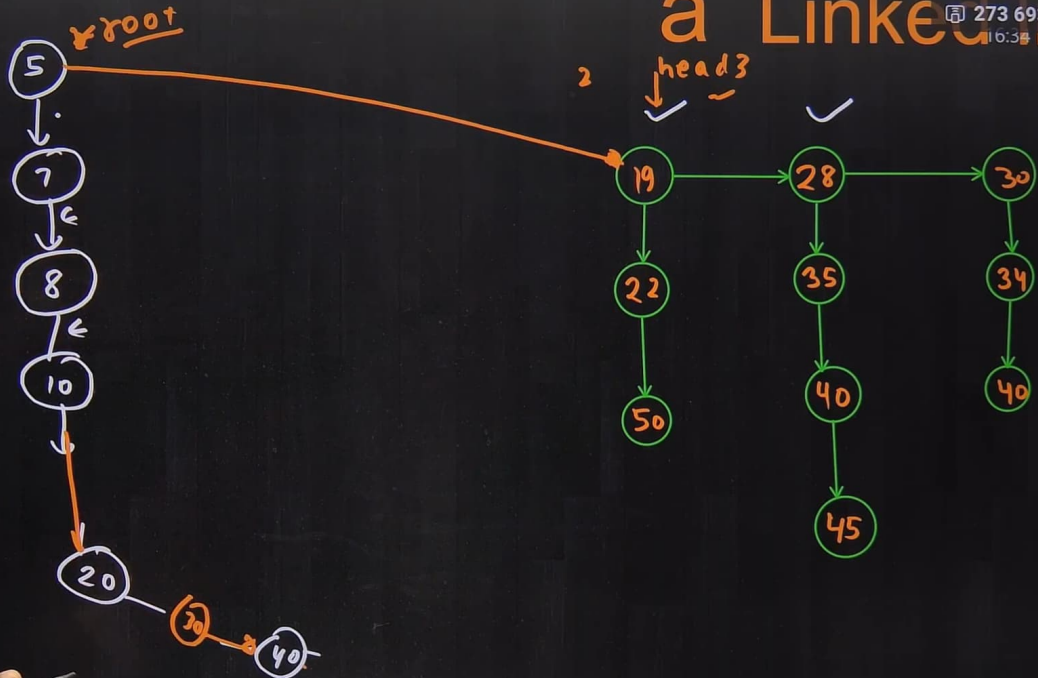
011 549
16:30



Flattening a Linked List

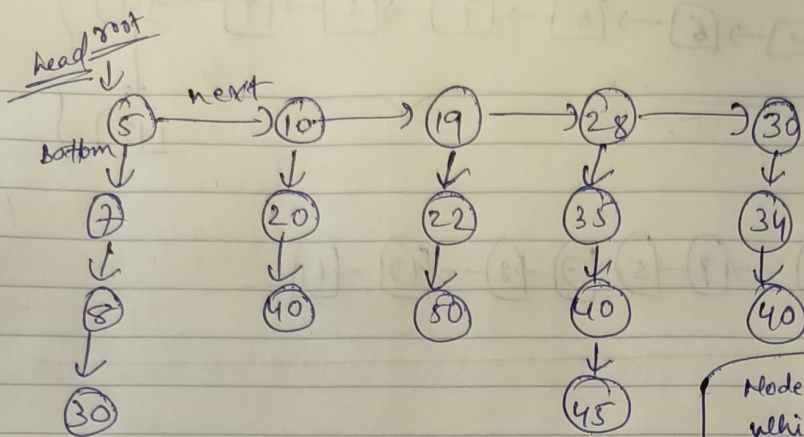







```
Node * head1, * head2, * head3;  
while (root->next)  
{  
    head1 = root;  
    head2 = root->next;  
    head3 = root->next->next;  
    head1->next = NULL;  
    head2->next = NULL;  
    root = merge(head1, head2);  
    root->next = head3;  
}
```


Flattening a LL



Subproblem: Merge 2 sorted LL.

head1 → 5 → 7 → 8 → 30

head2 → 10 → 20 → 40

Node * merge
funct
newhead
newtail
newtail

Node * head = new Node(0);
Node * newTail = newhead

while (head1 & head2) {

if (head1 → data < head2 → data) {

newTail → next = head1

head1 = head1 → bottom

newTail = newTail → bottom

newTail → bottom = NULL

}

else {

newTail → bottom = head2

head2 = head2 → bottom

newTail = newTail → bottom

newTail → bottom = NULL

}

}

```

Node * head1, * head2, * head3;
while (root → next) {
    head1 = root;
    head2 = root → next;
    head3 = root → next → next;
    head1 → next = NULL;
    head2 → next = NULL;
    root = merge (head1, head2);
    root → next = head3;
}
  
```

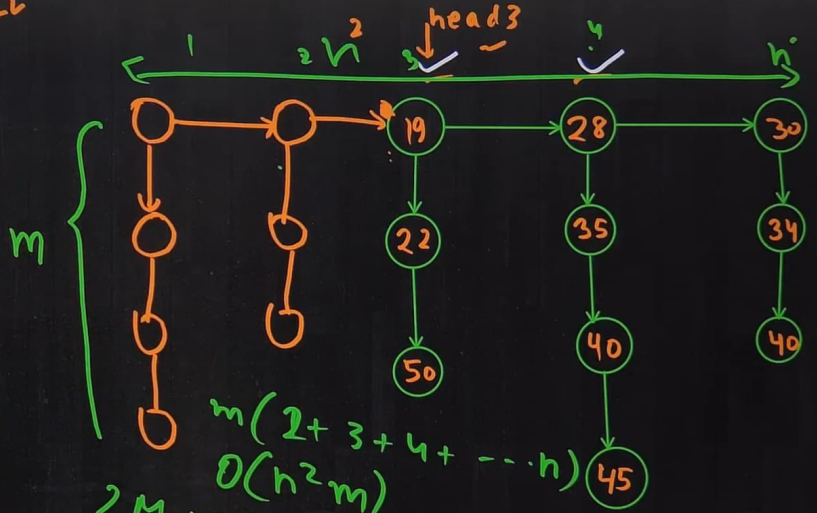
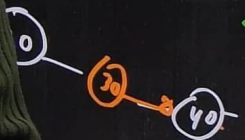
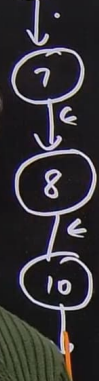
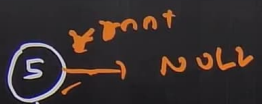
if (head1)

newTail → bottom = head1;

else

newTail → bottom = head2

return newhead → next;



$$m(2 + 3 + 4 + \dots + n)$$

$$O(n^2 m)$$

$$2M + 3M + 4M + \dots + nm$$